2.3.2 Making Restrictions on Patterns and Transformation Rules

When you use a "blank" _ in a pattern, it can stand for absolutely any expression. Often you will want to restrict the class of expressions that a particular blank can represent. There are several ways to do this.

The first way is to use objects like _*h* or *x*_*h* which stand for expressions which must have head *h*.

_	any expression
*x*_	any expression, to be named *x*
x_Integer	any integer, to be named *x*
x_Real	any approximate real number
x_Complex	any complex number
*x*_*h*	any object with head *h*

Some basic pattern objects.

This gives a transformation rule for the gamma function with an integer argument.

```
In[1]:= gamma[n_Integer] := (n-1)!
```

The rule applies to gamma with the integer argument 4, but not with the symbolic argument x.

```
In[2]:= gamma[4] + gamma[x]
Out[2]= 6 + gamma[x]
```

The rule does not apply to this expression because the object 4. has head Real, rather than Integer.

```
In[3]:= gamma[4.]
Out[3]= gamma[4.]
```

This defines a value for the function r when its first argument is a list, and its second argument is an integer.

```
In[4]:= r[list_List, n_Integer] := list^n
```

This matches the pattern in the rule defined for r.

```
In[5]:= r[{a, b}, 3]
                 3    3
Out[5]= {a , b }
```

This gives a rule for expressions with integer exponents.

```
In[6]:= d[x_^n_Integer] := n x^(n-1)
```

The rule is applied only in the cases with integer exponents.

```
In[7]:= d[x^4] + d[(a+b)^3] + d[x^(1/2)]

                3                  2
Out[7]= 4 x  + 3 (a + b)  + d[Sqrt[x]]
```

You can think of making an assignment for `f[x_Integer]` as like defining a function `f` that must take an argument of "type" `Integer`. In many cases, you will need to have more complicated restrictions on the arguments that can be given to a particular function. Often these restrictions correspond to the mathematical conditions for the validity of a particular formula.

Mathematica provides a general mechanism for making restrictions on the applicability of transformation rules. At the end of any definition that you give, you can add `/;` *condition*, where the `/;` can be read as "such that".

lhs `:=` *rhs* `/;` *condition*	define a transformation rule to be applied only if the condition is satisfied

Making a definition with a condition attached.

This gives a rule for `f` that applies only when its argument `n` is positive.

```
In[8]:= f[n_] := n! /; n > 0
```

The rule for `f` is used only with positive arguments.

```
In[9]:= f[6] + f[-4]
Out[9]= 720 + f[-4]
```

This gives another rule for `f` with a different condition.

```
In[10]:= f[n_] := fp[1-n] /; n < -1
```

This shows all the rules you have defined for `f`. The rules are inserted and tested in the order you give them. (If a new rule has *exactly* the same condition attached as an older one, the older one is overwritten.)

```
In[11]:= ?f
f
f/: f[n_] := n! /; n > 0
f/: f[n_] := fp[1 - n] /; n < -1
```

Here is a rule with a more complicated condition attached.

```
In[11]:= g[x_, y_] := pg[x, y] /; x > y+1 && x > 0
```

Mathematica cannot find out whether `a > 0` without knowing more about `a`. Unless a condition given with `/;` is explicitly `True`, *Mathematica* will not perform the transformation.

```
In[12]:= g[a, b]
Out[12]= g[a, b]
```

This is a common construction, implementing a feature of the "linearity" of a function `h`.

```
In[13]:= h[a_ b_, x_] := a h[b, x] /; FreeQ[a, x]
```

The terms independent of x are pulled out of each h.

```
In[14]:= h[2 (1+x) x^2, x] + h[a b x, x]

Out[14]= a b h[x, x] + 2 h[x^2 (1 + x), x]
```

There are many functions in *Mathematica* for testing properties of expressions. It is a convention that functions of this kind whose names end with the letter Q are set up to give False if their tests are not explicitly satisfied.

NumberQ[*expr*]	number
IntegerQ[*expr*]	integer
EvenQ[*expr*]	even number
OddQ[*expr*]	odd number
PrimeQ[*expr*]	prime number
PolynomialQ[*expr*, {x_1, x_2, ...}]	polynomial in x_1, x_2, ...
VectorQ[*expr*]	a list representing a vector
MatrixQ[*expr*]	a list of lists representing a matrix
OrderedQ[{*a*, *b*, ...}]	*a*, *b*, ... are in standard order
MemberQ[*expr*, *x*]	*x* is an element of *expr*
FreeQ[*expr*, *x*]	*x* appears nowhere in *expr*
MatchQ[*expr*, *form*]	*expr* matches the pattern *form*
ValueQ[*expr*]	a value has been defined for *expr*
AtomQ[*expr*]	*expr* has no subexpressions

Functions that test properties of expressions.

561 is an integer, so the test returns True.

```
In[15]:= IntegerQ[561]

Out[15]= True
```

This gives False, since x is not *known* to be an integer.

```
In[16]:= IntegerQ[x]

Out[16]= False
```

You can explicitly specify that x is an integer. Section 2.2.15 explains the meaning of the /: operator.

```
In[17]:= x/: IntegerQ[x] = True
Out[17]= True
```

This overrides the default assumption that x is not an integer.

```
In[18]:= IntegerQ[x]
Out[18]= True
```

This tests whether the elements of the list are in standard order.

```
In[19]:= OrderedQ[{a,b,d}]
Out[19]= True
```

The expression n is not a *member* of the list {x, x^n}.

```
In[20]:= MemberQ[{x, x^n}, n]
Out[20]= False
```

n does however appear *somewhere* in {x, x^n}.

```
In[21]:= FreeQ[{x, x^n}, n]
Out[21]= False
```

You can use /; conditions to set up patterns that match complicated classes of expressions. The essential idea is to have a basic pattern which matches a wide range of expressions, and then to use /; conditions to give complicated restrictions on the expressions.

This pattern matches only expressions that have the *structure* v[x_, 1 - x_].

```
In[22]:= v[x_, 1 - x_] := p[x]
```

This expression has the appropriate structure, and is matched.

```
In[23]:= v[a^2, 1 - a^2]
                 2
Out[23]= p[a ]
```

This expression does not have the appropriate structure, and so does not match. *Mathematica* does not, for example, try to "solve equations" to work out possible matches.

```
In[24]:= v[4, -3]
Out[24]= v[4, -3]
```

This pattern matches any expression w[x_, y_], with the added restriction that y == 1 - x.

```
In[25]:= w[x_, y_] := p[x] /; y == 1 - x
```

This expression matches the pattern. w[4, -3] is structurally of the form w[x_, y_]. In addition, the arguments 4 and -3 satisfy the condition y == 1 - x.

```
In[26]:= w[4, -3]
Out[26]= p[4]
```